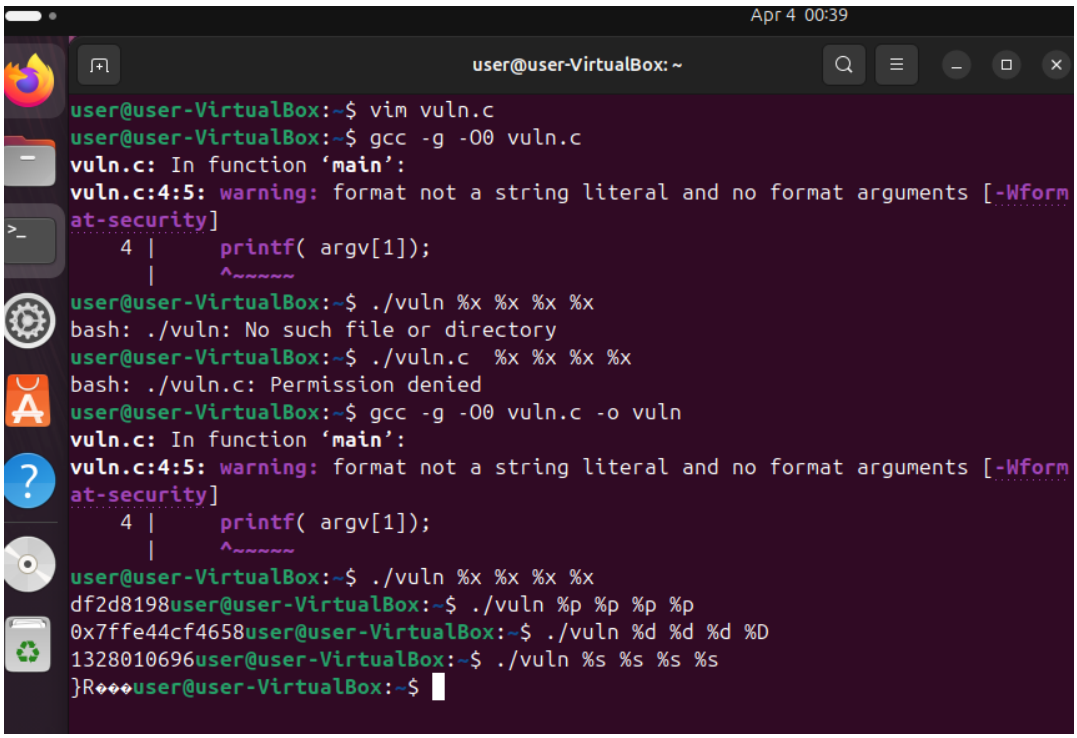
Beginner-C Bootcamp 3주차 과제

20215789 산업보안학과 김문정

Challenge 1. Format String Bug 실습



우선 gcc -g -O0 vuln.c이 무슨 명령어인지 몰라서 AI와 학습을 진행하였다. 먼저 -g 옵션은 디버깅 심볼을 포함한다는 의미로 gdb같은 디버거로 실행할 때 함수 이름, 변수 이름, 라인 번호 등을 확인 가능하도록 하는 명령어라는 것을 알 수 있었다. 또 -O0옵션은 컴파일러가 코드를 최적화하지 않도록 하여 스택에 코드에서 선언한 변수들이 그대로 존재하게끔 한다는 것임을 알 수 있었다.

또한 취약한 코드를 쓰고 컴파일할 때, 위 사진과 같은 경고가 나타나는 것을 확인할 수 있었다. printf에 문자열이 아닌 변수(argv[1])가 바로 들어간다는 포맷 문자열 취약점 가능성을 컴파일러가 감지했다는 것이다. 위 경고를 무시하고 ./vuln %x %x %x %x, %p, %s, %d등의 형태로 실행해서 출력값을 관찰하면 위 사진과 같은 값이 나오는 것을 확인할 수 있다.

%x → 스택에 있던 값 하나를 16진수로 읽고,

%p → 스택에 있던 주소값 출력

%d → 스택에 있는 값을 10진수로 해석

%s → 스택에 있는 주소를 문자열로 출력

위와 같은 서식지정자의 출력이 무엇을 뜻하는지도 확인할 수 있었다.

취약한 코드에서 (printf("%s", argv[1]))로 수정하여 다시 컴파일을 진행하였더니 아래와 같은 결과가 나오는 것을 확인하였다.

```
user@user-VirtualBox:~$ vim vuln.c
user@user-VirtualBox:~$ gcc -g -O0 vuln.c -o vuln
user@user-VirtualBox:~$ ./vuln %x %x %x %x
%xuser@user-VirtualBox:~$ ./vuln %p %p %p %p
%puser@user-VirtualBox:~$ ./vuln %s %s %s %s
%suser@user-VirtualBox:~$
```

먼저 printf(argv[1]); 을 하고 실행할 때 %x을 넣어서 실행시키면, 사용자의 입력값이 명령어의 역할을 하게 되어 스택에 있던 값을 읽어오게 된다. 그러나 printf("%s", argv[1]); 로 바꾸어 실행시키면, %s 서식지정자가 argv[1]에 들어오는 값을 문자열로 해석하기 때문에 %x, %p와 같이 문자열로 동작한다는 것을 알 수 있었다.

Challenge2. 반복문 3종 비교

```
user@user-VirtualBox: ~                    user@user-Virt
#include <stdio.h>

int main(){

        int sum=0;
        for (int i=1; i<=10; i++){
        sum += i;
    }

    printf("sum = %d\n", sum);
}
~
~
```

```
user@user-VirtualBox: ~                    user@user-VirtualBox: ~
#include <stdio.h>

int main(){
        int sum=0;
        int num=1;

        while(num<11)
        {       sum+=num;
                num++;

        }

        printf("sum = %d\n", sum);


}
~
~
~
```

```c
#include <stdio.h>

int main(){
        int sum=0;
        int i=0;
        do{
                sum+=i;
                i++;
        }while(i<=10);

        printf("sum = %d\n", sum);

}
~
~
```

```
user@user-VirtualBox:~$ vim c_files/for.c
user@user-VirtualBox:~$ vim c_files/while.c
user@user-VirtualBox:~$ gcc -g -O0 c_files/while.c -o while
user@user-VirtualBox:~$ ./while
sum = 55
user@user-VirtualBox:~$ vim c_files/while.c
user@user-VirtualBox:~$ vim c_files/dowhile.c
user@user-VirtualBox:~$ gcc -g -O0 c_files/dowhile.c -o dowhile
user@user-VirtualBox:~$ ./dowhile
sum = 55
user@user-VirtualBox:~$ vim c_files/dowhile.c
user@user-VirtualBox:~$
```

각 반복문으로 구현했고 컴파일 또한 완료하였다.

for문은 for(초기식;조건식;증감식){명령문}으로 구성되고, while문은 while(조건식){명령문;조건식 조절;}의 형식, do while문은 do{명령문;조건식 조절;}while(조건식); 의 형태를 띈다는 것을 학습하였다.

for문은 반복 횟수가 정해져 있을 때 사용하기 용이하고, while문은 언제 끝날지 모를 때, do while문은 조건을 먼저 검사하고 만족하지 않으면 실행하지 않는 위 두개와는 달리 최소한 한 번이라도 실행이 필요할 때 용이하다는 것을 알 수 있었다.

Challenge3. 조건문으로 분기 구현

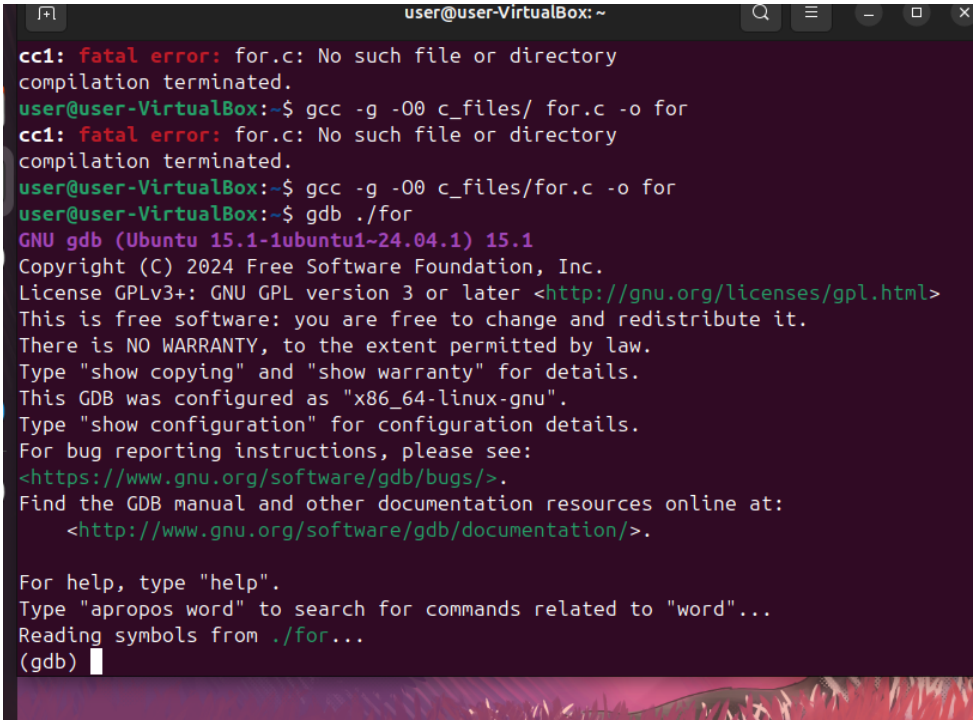
```c
#include <stdio.h>

int main() {
    int num;
    printf("enter an integer: ");
    scanf("%d", &num);

    if (num > 0) {
        printf("positive\n");
    } else if (num < 0) {
        printf("negative\n");
    } else {
        printf("0\n");
    }

}
~
```

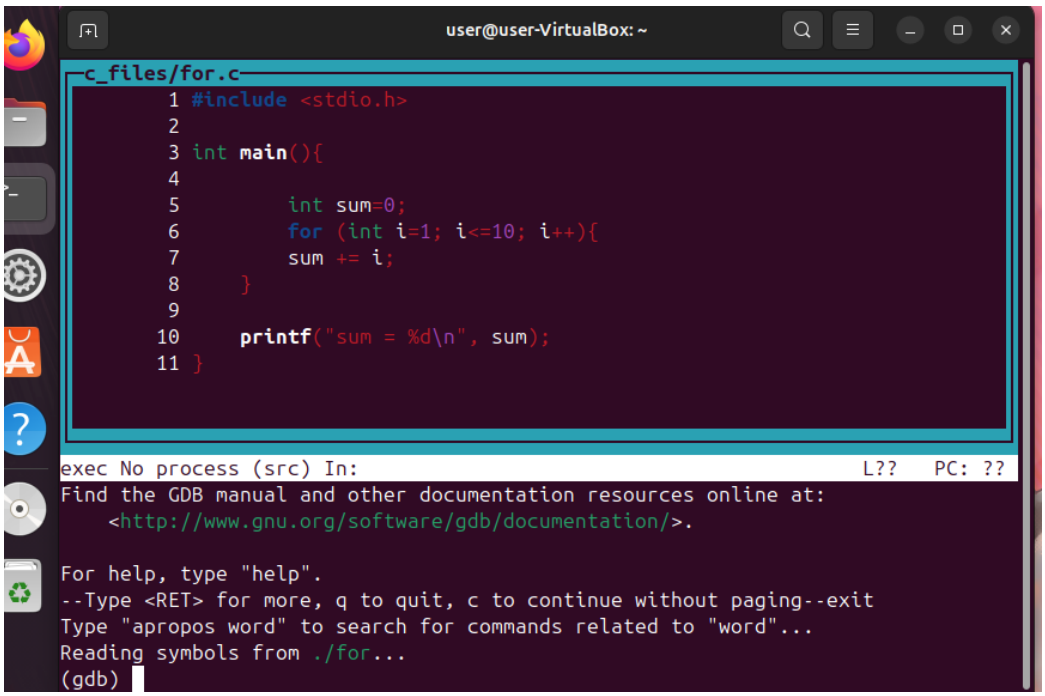
```c
#include <stdio.h>

int main() {
    int num;
    printf("enter an integer: ");
    scanf("%d", &num);

    printf("%s\n", (num > 0) ? "positive" : (num < 0) ? "negative" : "0");

}
~
~
```

```c
int main()
{
 int a;

 printf("enter an integer:");
 scanf("%d", &a);

 if (a > 0)
  a = 1;

 else if (a < 0)
  a = 2;

 else
  a = 0;

 switch (a)
 {
 case 1 :
  printf("positive\n");
  break;
 case 2 :
  printf("nagative\n");
-- INSERT --                                   7,2          14%
```

switch문으로 구현할 때는 case에 양수밖에 작성하지 못하므로 앞에서 양수면 a=1, 음수면 a=2가 되도록 조건을 미리 작성했다.

Challenge4. gdb로 반복문 뜯어보기





gdb -tui ./for를 하면 위 사진과 같이 소스코드를 보며 컴파일을 진행할 수 있다. break 6을 걸고 run을 하면 해당 breakpoint에서 멈추는 것을 알 수 있다. next와 step로 실행하면 반복문의 동작 순서를 볼 수 있다.

```
c_files/for.c
        3 int main(){
        4
        5         int sum=0;
B+>     6         for (int i=1; i<=10; i++){
        7         sum += i;
        8     }
        9
       10     printf("sum = %d\n", sum);
       11 }

multi-thre Thread 0x7ffff7fa97 (src) In: main          L6    PC: 0x55555555516b
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 2, main () at c_files/for.c:6
(gdb) step
(gdb) step
(gdb) next
(gdb) next
(gdb)
```

```
c_files/for.c
        3 int main(){
        4
        5         int sum=0;
B+>     6         for (int i=1; i<=10; i++){
        7         sum += i;
        8     }
        9
       10     printf("sum = %d\n", sum);
       11 }

multi-thre Thread 0x7ffff7fa97 (src) In: main          L6    PC: 0x55555555
rax            0x2                 2
rbx            0x7fffffffdf28      140737488346920
rcx            0x555555557dc0      93824992247232
rdx            0x7fffffffdf38      140737488346936
rsi            0x7fffffffdf28      140737488346920
rdi            0x1                 1
rbp            0x7fffffffde00      0x7fffffffde00
--Type <RET> for more, q to quit, c to continue without paging--
```

또한 info registers명령을 사용하여 레지스터의 값들도 읽어올 수 있다.